Linear Regression to Deep Learning Foundations

Compact reference guide - starts after the calculus PDF

Starting point: You already covered derivatives, integrals, chain rule, gradient descent, and backpropagation in the attached calculus PDF. This guide starts from the next learning items and keeps explanations short for future doubt clearance.

1. Learning Order

Order	Topic	Purpose
1	Linear Regression	Predict one continuous number using one input.
2	Loss Function	Measure prediction error.
3	Derivatives of Loss	Find how m and b affect loss.
4	Gradient Descent	Update parameters to reduce loss.
5	Multiple Linear Regression	Use many input features.
6	Linear Algebra	Use vectors, matrices, shapes, dot product.
7	Matrix Multiplication	Compute many predictions or neurons efficiently.
8	Neural Network	Many weighted sums with activation functions.

2. Linear Regression

Linear Regression predicts a continuous value such as price, salary, marks, temperature, or sales.

y_hat = m*x + b

Term	Meaning
x	Input feature. Example: house size.
y	Actual answer from dataset.
y_hat	Predicted answer from model.
m	Slope/weight. Controls impact of x.
b	Bias/intercept. Shifts prediction up or down.

Example: If y_hat = 2*x + 1 and x = 5, prediction is 11.

3. Cost / Loss Function

Loss says how wrong the model is. Lower loss means better prediction.

Single example: Loss = (y_hat - y)^2

Dataset: MSE = (1/n) * sum((y_hat - y)^2)

- Squaring removes negative signs.
- Big mistakes get bigger penalty.
- MSE is the common loss for Linear Regression.

4. Derivatives of the Loss

Derivatives tell whether to increase or decrease parameters.

y_hat = m*x + b

Loss = (m*x + b - y)^2

dLoss/dm = 2*(y_hat - y)*x

dLoss/db = 2*(y_hat - y)

Derivative sign	What it means
Positive	Gradient descent decreases the parameter.
Negative	Gradient descent increases the parameter.
Zero	No major update is needed.

5. Gradient Descent

Gradient Descent updates parameters opposite to the derivative direction.

m = m - learning_rate * dLoss/dm

b = b - learning_rate * dLoss/db

- learning_rate controls step size.
- Too small: slow training.
- Too large: loss may explode or jump.
- Epoch: one full pass through all training data.

Core loop: Predict -> Loss -> Derivatives -> Update -> Repeat.

6. Simple Linear Regression - Python from Scratch

This code uses only Python lists. It shows every calculation clearly.

```python
def predict(x, m, b):
    return m * x + b

def mse_loss(X, Y, m, b):
    total = 0
    n = len(X)
    for x, y in zip(X, Y):
        error = predict(x, m, b) - y
        total += error**2
    return total / n

def gradients(X, Y, m, b):
    dm = 0
    db = 0
    n = len(X)

    for x, y in zip(X, Y):
        error = predict(x, m, b) - y
```

```
        dm += 2 * error * x
        db += 2 * error

    return dm / n, db / n

def train_linear_regression(X, Y, learning_rate=0.01, epochs=1000):
    m = 0
    b = 0

    for epoch in range(epochs):
        dm, db = gradients(X, Y, m, b)

        m = m - learning_rate * dm
        b = b - learning_rate * db

        if epoch % 100 == 0:
            loss = mse_loss(X, Y, m, b)
            print(
                f"Epoch {epoch} | "
                f"Loss {loss:.6f} | "
                f"m {m:.4f} | "
                f"b {b:.4f}"
            )

    return m, b


# Training Data
X = [1, 2, 3, 4]
Y = [3, 5, 7, 9]  # Expected pattern: y = 2*x + 1
# Train the model
m, b = train_linear_regression(X, Y)
# Final result
print("Final Weights:")
print(f"m = {m:.4f}")
print(f"b = {b:.4f}")
# Test prediction
print(f"Prediction for x = 5 : {predict(5, m, b):.4f}")
```

Expected result: m becomes close to 2, b becomes close to 1, and prediction for x=5 becomes close to 11.

7. How the Model Learns Over Time

Stage	What happens
Start	m and b are poor, so predictions are poor.
Early epochs	Loss usually decreases quickly.
Later epochs	Loss decreases slowly because model is near best line.
End	Stop when loss is low or no longer improving.

- Loss decreasing means training is working.

- Loss increasing often means learning rate is too high or data needs scaling.
- Loss stuck can mean learning rate is too small or model is too simple.

8. Multiple Linear Regression

Simple Linear Regression uses one feature. Multiple Linear Regression uses many features.

```
y_hat = w1*x1 + w2*x2 + w3*x3 + ... + b
```

Simple Linear Regression	Multiple Linear Regression
One input x	Many inputs: x1, x2, x3, ...
One weight m	One weight for each feature.
y_hat = m*x + b	y_hat = w1*x1 + w2*x2 + ... + b
Example: price from size	Example: price from size, bedrooms, age

Example house price model:

```
Price = w1*Size + w2*Bedrooms + w3*Age + b
```

9. Linear Algebra Basics

Concept	Simple meaning	ML example
Scalar	One number	learning_rate = 0.01
Vector	List of numbers	[size, bedrooms, age]
Matrix	Table of numbers	Rows = houses, columns = features
Tensor	General number container	Images, batches, model inputs
Shape	Dimension information	(1000,20) = 1000 rows, 20 features

Dataset matrix example:

```
X = [
    [1.0, 2, 10],   # house 1: size, bedrooms, age
    [1.5, 3, 5],    # house 2
    [2.0, 4, 2],    # house 3
]
```

10. Dot Product and Matrix Multiplication

Dot product means multiply matching values and add the results.

[2, 3, 4] dot [5, 6, 7] = 2*5 + 3*6 + 4*7 = 56

For one row, dot product gives one prediction. For many rows, matrix multiplication gives many predictions.

Predictions = X*W + b

- X is the input matrix.
- Rows of X are samples.
- Columns of X are features.
- W is the weight vector or weight matrix.
- Columns of X must match rows of W.

Shape rule: (m x n) * (n x p) = (m x p)

Inside numbers must match. Outside numbers become the output shape.

11. Multiple Linear Regression Derivatives

Each weight has its own derivative.

dLoss/dwj = 2*(y_hat - y)*xj

dLoss/db = 2*(y_hat - y)

- wj is the weight for feature j.
- xj is the value of feature j in the current row.
- Bias has one derivative because there is one bias term.

12. Multiple Linear Regression - Python from Scratch

This is the direct extension of simple linear regression.

```python
def prediction(features, weights, bias):
    y_hat = 0

    for x, w in zip(features, weights):
        y_hat += x * w

    return y_hat + bias


def compute_loss(inputs, outputs, weights, bias):
    total = 0
    n = len(inputs)

    for i in range(n):
        error = prediction(inputs[i], weights, bias) - outputs[i]
        total += error ** 2

    return total / n


def gradient_calculation(inputs, outputs, weights, bias):
    n = len(inputs)
    number_of_features = len(weights)

    dw = [0] * number_of_features
    db = 0

    for i in range(n):
        y_hat = prediction(inputs[i], weights, bias)
        error = y_hat - outputs[i]

        for j in range(number_of_features):
            dw[j] += 2 * error * inputs[i][j]

        db += 2 * error

    for j in range(number_of_features):
        dw[j] = dw[j] / n

    db = db / n

    return dw, db


def train_multiple_linear_regression(
    inputs,
    outputs,
```

```
    learning_rate=0.01,
    epochs=5000
):
    number_of_features = len(inputs[0])

    weights = [0] * number_of_features
    bias = 0

    for epoch in range(epochs):
        dw, db = gradient_calculation(inputs, outputs, weights, bias)

        for j in range(number_of_features):
            weights[j] = weights[j] - learning_rate * dw[j]

        bias = bias - learning_rate * db

        if epoch % 500 == 0:
            loss = compute_loss(inputs, outputs, weights, bias)

            print(
                f"Epoch {epoch} | "
                f"Loss {loss:.6f} | "
                f"Weights {weights} | "
                f"Bias {bias:.4f}"
            )

    return weights, bias


# ----------------------------
# Training Data
# Features: [size_in_1000_sqft, bedrooms, age]
# ----------------------------
inputs = [
    [1.0, 2, 10],
    [1.5, 3, 5],
    [2.0, 4, 2],
    [2.5, 4, 1]
]

# Output is scaled price
# 2.0 means 200000
outputs = [2.0, 3.0, 4.0, 5.0]


# ----------------------------
# Train Model
# ----------------------------
weights, bias = train_multiple_linear_regression(inputs, outputs)
```

```
# ----------------------------
# Test Prediction
# ----------------------------
new_house = [1.8, 3, 4]

scaled_price = prediction(new_house, weights, bias)
real_price = scaled_price * 100000

print("Predicted price:", real_price)
```

13. Why Scaling Matters

Gradient descent works better when inputs and outputs are not extremely large.

Raw value	Scaled value
1000 sq ft	1.0
200000 price	2.0
2500 sq ft	2.5
500000 price	5.0

- Large values create large gradients.
- Large gradients can make loss explode.
- Scaling allows a reasonable learning rate such as 0.01.

14. Connection to Deep Learning

A neuron before activation is almost the same as Multiple Linear Regression.

weighted_sum = w1*x1 + w2*x2 + ... + b
neuron_output = activation(weighted_sum)

Classical ML idea	Deep learning idea
Linear Regression	One neuron without activation
Multiple Linear Regression	One neuron with many inputs
Matrix Multiplication	Computes all neurons in a layer
Gradient Descent	Updates neural-network weights
Backpropagation	Chain rule through many layers

Deep Learning is the same learning loop applied to many layers, many weights, and usually nonlinear activation functions.

15. Final Cheat Sheet

Question	Answer
What does Linear Regression predict?	A continuous number.
What is y_hat?	The model prediction.
What is loss?	How wrong the prediction is.
Why derivatives?	To know how changing each parameter changes loss.
What is Gradient Descent?	Update parameters opposite to the gradient.
What is an epoch?	One full pass over training data.
What is a vector?	A list of numbers.

What is a matrix?	A table of numbers.
What is dot product?	Multiply matching values and add them.
Why matrix multiplication?	To compute many predictions or neurons efficiently.

16. Formula Cheat Sheet

Simple LR: y_hat = m*x + b

Multiple LR: y_hat = w1*x1 + w2*x2 + ... + b

MSE = (1/n) * sum((y_hat - y)^2) dLoss/dm = 2*(y_hat - y)*x

dLoss/dwj = 2*(y_hat - y)*xj

dLoss/db = 2*(y_hat - y)

parameter = parameter - learning_rate * gradient

17. Next Study Step

Next recommended chapter: Neurons and the First Neural Network.

- Understand one neuron as weighted sum plus activation.
- Understand one layer as many neurons together.
- Understand a neural network as many layers connected together.
- Implement a tiny neural network from scratch before using PyTorch.

18. Forward and Backward Pass

Forward Propagation - It takes the input, multiplies it by the weights, adds the bias, passes it through the activation function (if there is one), and produces the prediction $\hat{y}$.

For **Linear Regression**, there is **no activation function**.

Backward Propagation is the process of calculating how much each weight and bias contributed to the prediction error by computing the gradients (derivatives) of the loss function.

FORWARD PASS	BACKWARD PASS
Input (x) | ▼ Multiply by Weights | ▼ Add Bias | ▼ Activation (if any) | ▼	Prediction (ŷ) | ▼ Compare with Actual (y) | ▼ Calculate Loss | ▼ Calculate Gradients (∂Loss/∂Weights, ∂Loss/∂Bias) |

Prediction (ŷ)	▼ Gradient Descent | ▼ Update Weights & Bias